
Formal Verification Update Report

Manifest — Global Orders, Order Types & Seat Integrity

July 2026

Prepared for Manifest

This report documents an extension of the existing formal verification of Manifest. It follows the structure of the original third-party assessment but was produced independently of it.

Project Summary

Project Scope

Project name	Repository	Date	Platform
Manifest	github.com/Bonasa-Tech/manifest	July 2026	Solana

Files in scope for this update: the market matching engine (`state/market.rs`, `state/market_helpers.rs`), the global order account (`state/global.rs`, `state/utlis.rs`), the resting order type (`state/resting_order.rs`), the global deposit and withdraw processors, and the verification harness and rule specifications in the repository.

Project Overview

The original verification of Manifest established the “no loss of funds” invariant — `vault == withdrawable + orderbook`, for base and quote — for limit-order flows, with global orders explicitly disabled in the verified build and several verification-only stubs in place. This update removes those stubs and extends the verified surface to:

- **Global orders**, whose funds come from the global vault rather than the market vault. A new invariant, `global_vault == sum of all global deposits`, is introduced and shown to be preserved jointly with the market invariant through matching (full, partial, unbacked), resting, cancelling, deposits, withdrawals and eviction.
- **All other order types**: post-only and global takers (which must never take), immediate-or-cancel, and reverse / reverse-tight orders including the coalescing of a come-back order into an existing one.
- **Gas prepayments** for global orders, as lamport-conservation properties.
- **Seat pubkey integrity**: the pubkey stored in a claimed seat is written by `claim_seat` and never mutated while the seat is held — the invariant on which the attribution of global funds rests.

All 52 new rules and all 49 pre-existing rules were checked on the updated code with an SMT-based prover for SBF binaries; per-rule status appears in the Formal Verification section. During the work, one production accounting defect (F-01) and five verification-integrity defects (F-02-F-06) were found and fixed.

Findings Summary

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	1	1	1

Informational	5	5	5
Total	6	6	6

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
	Likelihood			

Detailed Findings

ID	Title	Severity	Status
F-01	Reverse order coalescing debits the maker more than the order can refund	Low	Fixed in this change
F-02	Global orders were excluded from formal verification by a build-time stub	Informational	Fixed in this change
F-03	Verification build of cancel_order_by_index returned early for global orders	Informational	Fixed in this change
F-04	The one-step matching model had drifted from the production matching loop	Informational	Fixed in this change
F-05	Mock seat release never released the second seat	Informational	Fixed in this change
F-06	Orderbook ghost counters not maintained on the reverse coalesce path	Informational	Fixed in this change

F-01. Reverse order coalescing debits the maker more than the order can refund

Severity: **Low** **Status:** Fixed in this change

When a filled reverse order is placed back on the other side of the book and an order of the same maker already rests at the come-back price, the new size is coalesced into the existing order. The maker was debited `ceil(size × price_reverse)`, but the coalesced order's refundable allocation — what it returns when cancelled, computed as a whole at its own price `q` — only grows by `ceil((old + size) × q) - ceil(old × q)`. The two differ because the rounding of the combined allocation does not split cleanly, and because coalescing tolerates a one-increment price difference (`RestingOrder::eq`), in which case `q ≠ price_reverse`.

Worked example (from `reverse_order_tight_type_test`): a reverse-tight ask of 3 SOL at 3.0 with a 0.05% spread comes back as a bid at 2.9985. A first fill of 1 SOL pays the maker 3,000,000,000 quote atoms; the come-back bid rests fresh with 1,000,500,250 base atoms and debits $\text{ceil}(1,000,500,250 \times 2.9985) = 3,000,000,000$ — exact. A second fill of 2 SOL pays 6,000,000,000; the come-back size 2,001,000,500 coalesces, growing the resting bid to 3,001,500,750 atoms, whose allocation is $\text{ceil}(3,001,500,750 \times 2.9985) = 8,999,999,999$ — a growth of 5,999,999,999. The old code debited 6,000,000,000: the maker paid 9,000,000,000 in total for an order that can only ever refund 8,999,999,999, permanently stranding one atom in the vault. In the price-tolerance case the sign can flip and the order becomes under-backed at the expense of the vault, by up to size $\times$ one price increment per coalesce.

Impact: dust-level value per event, but it breaks the exact invariant `vault == withdrawable + orderbook` in both directions, and the under-backed direction is repeatable.

Fix: on coalesce, debit the maker the exact growth of the coalesced order's allocation, computed at that order's own price, so the debit always matches what the order will refund. The added size is additionally capped so the debit never exceeds the quote the maker received from that fill: growing the order by the full requested size can round up to more quote than the proceeds cover, which would otherwise force the maker to top up from other balances, or, with none, revert an otherwise valid fill (exercised by `reverse_order_coalesce_at_higher_price_does_not_overdraw_proceeds_test`). The fresh-rest path already rounds the size down and is unchanged. Where the exact growth is already within the proceeds (the worked example above), the cap does not engage and the debit is unchanged; the affected test expectation there moved by one atom, the maker keeping the atom that used to strand in the vault.

F-02. Global orders were excluded from formal verification by a build-time stub

Severity: **Informational** **Status:** Fixed in this change

Under the verification feature, `RestingOrder::is_global()` was hardcoded to `false`. Every global branch in matching, resting and cancelling was dead code in the verified binary, and the global account was not modeled at all, so previous “no loss of funds” results said nothing about global orders. The stub was removed, the global account is now modeled (two seats, real balance code, mocked tree plumbing), and the properties in P-01–P-08 cover the global flows.

F-03. Verification build of `cancel_order_by_index` returned early for global orders

Severity: **Informational** **Status:** Fixed in this change

The verification build of `cancel_order_by_index` returned before removing a cancelled global order from the book — a divergence from production that was known and annotated in the code, tolerated because global orders were stubbed out (F-02). With global orders modeled, the early return was removed; the verified code now matches production.

F-04. The one-step matching model had drifted from the production matching loop

Severity: Informational **Status:** Fixed in this change

`place_single_order`, the verification model of one iteration of `place_order`'s matching loop, had three divergences on the global path: base and quote were swapped when drawing down a global maker's deposit; the handling of a missing global account (halt for a taker, fail for a resting order) was absent; and the rounding bonus atom was credited to global makers, for whom production explicitly skips it. It also lacked the reverse-order re-placement entirely. The model now mirrors the production loop body, and the divergence risk is documented as a known limitation.

F-05. Mock seat release never released the second seat

Severity: Informational **Status:** Fixed in this change

`release_second_seat` in the market mock set the occupancy flag to 1 (taken) instead of 0 (free), so no verified path could ever observe the second seat being released.

F-06. Orderbook ghost counters not maintained on the reverse coalesce path

Severity: Informational **Status:** Fixed in this change

The verification-only aggregates `orderbook_base_atoms` / `orderbook_quote_atoms` were not updated when a reverse order coalesced into an existing order, in either the production function or the model. Harmless on-chain (the fields are compiled out), but it would have surfaced as a phantom invariant violation once reverse orders became reachable in verification.

Formal Verification

Verification Notations

Verified — the rule is verified for every state of the program, under the assumptions and simplifications stated below.

Verified after fix — the rule was violated, the cause was fixed (in the code or in a missing modeled invariant, as described per rule), and the rule verifies afterwards.

General Assumptions and Simplifications

1. Mocks carried over from the original verification setup are kept: the market's red-black trees are replaced by a two-trader, one-order-per-side abstraction, and prices are single-limb with four decimal places.
2. A new mock models the global account with two seats, owned by the same two traders the market mock knows. Lookups scan the pubkeys stored in the nodes (so eviction,

- which hands a slot to a new trader, is expressible). All balance-touching code is the real implementation; only the tree plumbing is mocked.
3. The stub that disabled global orders (`is_global` always `false`) is removed; global branches are verified as written.
 4. SPL Token and Token-2022 transfers are summarized as direct moves of the token-account amounts. Transfer fees are not modeled, so `received-amount-differs-from-requested` handling is verified in the fee-less case only.
 5. `checked_multiply_rational` (reverse order come-back prices) is implemented exactly on the single-limb price, so the same inputs give the same price inside and outside the matching code.
 6. `require!` compiles to an assumption under the verification feature: production failure paths appear as infeasible paths. “Must fail” properties are therefore stated as “no reachable execution does X”.
 7. Whether a token-2022 mint carries a transfer fee or hook depends on mint state that is not modeled; the decision “treat this global order as unbacked” is over-approximated by a nondeterministic choice at the exact program point of the production checks.
 8. The system-program transfer of the gas prepayment is summarized as a direct `lamport` move.
 9. The claimed-seat nodes are assumed to carry the pubkeys of the traders sitting in them. This assumption is discharged by the P-12 rules.
 10. Assumptions that relate two pieces of mocked state to each other (for example, that a resting order's price sits within one increment of another price) are written against the stored fields. Equating a whole value with a constructed one does not propagate the relation into the mock's memory, and the code then re-reads a value unrelated to the one assumed. Rules that only havoc a field, by equating it with a fresh nondeterministic value, are unaffected.
 11. The reverse spread is a `u16` field read out of mocked memory; the prover does not recover that width, so the rules bound it to what the type can hold. Without the bound ``base - spread`` in `reverse_price` underflows.
 12. `place_single_order` is the model of one iteration of the matching loop in `place_order` and is kept behaviourally identical to it by hand; this remains a manually maintained correspondence.

Regression: pre-existing properties

All 49 rules from the original verification (deposits, withdrawals, resting, cancelling, matching balance deltas, market and seat management, batch update integrity, and swaps) were re-run against the updated code and remain **Verified**. The four rules that pin down the exact balance deltas of a plain match now additionally assume a non-reverse maker, because a reverse maker legitimately moves funds again when its come-back order is placed; the no-loss-of-funds coverage of reverse makers is provided by P-11.

Formal Verification Properties

P-01. “no loss of funds” for global orders - matching, full fill

Status: Verified after fix

A global order is not backed by the market vault. The maker's tokens sit in the global vault, a token account shared by every market that trades the mint, and are pulled into the market vault at the moment the order is matched. The global vault is therefore a second, independent source of funds with its own invariant: `global_vault == sum of all global deposits`, tracked by a new ghost field `global_deposited_atoms` on `GlobalFixed`. These rules match a taker order against a resting global maker order that is fully consumed, and assert: both the market invariant and the global invariant are preserved; the amount drawn from the maker's global deposit equals the traded amount on the side the maker backs; and exactly that amount moves from the global vault to the market vault.

Rule name	Status	Description
<code>rule_place_single_order_global_full_match_bid</code>	Verified after fix	Taker bid fully fills a global ask backed by base atoms. This rule was violated until the seat-pubkey invariant was made explicit: the prover found a market in which the maker's claimed-seat node carried a different trader's pubkey, so the trade drew down another trader's global deposit. That state is unreachable in a real market (<code>claim_seat</code> writes the key and nothing else mutates it, see P-12), and the invariant is now a precondition.
<code>rule_place_single_order_global_full_match_ask</code>	Verified after fix	Taker ask fully fills a global bid backed by quote atoms. Same counter-example and fix as the bid variant.

P-02. “no loss of funds” for global orders - matching, partial fill

Status: Verified after fix

Same setting as P-01, but the taker is exhausted before the maker order is fully consumed, so the global order stays on the book with reduced size. Quote rounding is in the maker's favor on partial fills.

Rule name	Status	Description
<code>rule_place_single_order_global_partial_match_bid</code>	Verified after fix	Taker bid partially fills a global ask. Same seat-pubkey counter-example and fix as P-01.
<code>rule_place_single_order_global_partial_match_ask</code>	Verified after fix	Taker ask partially fills a global bid. Same seat-pubkey counter-example and fix as P-01.

P-03. “no loss of funds” for global orders - unbacked orders

Status: **Verified**

A global order whose maker can no longer cover it is removed from the book without trading. The removal must not move a single atom: the maker was never debited on the market, so nothing may be refunded, and the balance check happens before the maker's global deposit is reduced, so a rejected fill must not eat the deposit. The token-2022 transfer-fee and transfer-hook rejections return through the same point and are covered by the same rules (see assumption 7).

Rule name	Status	Description
rule_place_single_order_global_unbacked_bid	Verified	A taker bid meets a global ask that cannot cover the trade. The order is removed, nothing trades, no market or global balance changes.
rule_place_single_order_global_unbacked_ask	Verified	A taker ask meets a global bid that cannot cover the trade. Same assertions.

P-04. “no loss of funds” for global orders - rest_remaining

Status: **Verified**

Resting a global order reserves nothing on the market: the funds stay in the global account. The rules assert that no market balance, vault balance, or global balance moves, and in particular that the orderbook aggregate does not grow, which is what keeps the market invariant and the global invariant independent.

Rule name	Status	Description
rule_rest_remaining_global_bid	Verified	rest_remaining inserts a global bid into the market. No funds move anywhere.
rule_rest_remaining_global_ask	Verified	rest_remaining inserts a global ask into the market. No funds move anywhere.

P-05. “no loss of funds” for global orders - cancel

Status: **Verified**

Cancelling a global order gives nothing back on the market, because nothing was taken when it was placed, and leaves the global account untouched. The maker in particular is not credited for an order they never funded from the market. A prior verification-build-only early return in `cancel_order_by_index` (finding F-03) skipped the removal of the order from the book in the verified model; it was removed as part of this work.

Rule name	Status	Description
rule_cancel_global_order_bid	Verified	Cancels a resting global bid. Market and global balances are unchanged; the order is removed.

rule_cancel_global_order_ask	Verified	Cancels a resting global ask. Same assertions.
------------------------------	----------	--

P-06. “no loss of funds” for the global account - deposit and withdraw

Status: Verified

Deposits and withdrawals on the global account are the analogue of P-01/P-02 of the original report for the global vault: the vault and the depositor's balance move in lock-step, and a trader can never withdraw more than they deposited.

Rule name	Status	Description
rule_global_deposit	Verified	Depositing to the global account moves exactly the deposited atoms from the trader's token account into the global vault and credits the depositor for exactly that.
rule_global_withdraw	Verified	Withdrawing moves exactly the requested atoms from the global vault to the trader, reduces the depositor's balance by the same, and cannot exceed the deposit.

P-07. global seat eviction

Status: Verified

Evicting the global trader with the smallest (zero) deposit hands their seat to a new trader without moving tokens. Verified at the state level (evict_and_take_seat); the surrounding processor (seat fee, eviction fee, token transfers) is out of scope.

Rule name	Status	Description
rule_global_evict	Verified	After eviction the new trader holds the seat with a zero balance, the evictee holds no seat, and the global vault and deposit aggregate are unchanged.

P-08. gas prepayment conservation

Status: Verified

Every global order deposits GAS_DEPOSIT_LAMPORTS on the global account when placed; whoever removes the order claims the refund. These are lamports, not tokens, and get their own conservation rules. The system-program transfer is summarized as a direct lamport move.

Rule name	Status	Description
rule_global_gas_prepayment	Verified	Placing n global orders moves exactly n * GAS_DEPOSIT_LAMPORTS from the gas payer to the global account.

rule_cancel_global_order_gas_refund_bid	Verified	Cancelling a global bid with the system program present pays exactly one refund from the global account to the gas receiver; token balances are untouched.
rule_cancel_global_order_gas_refund_ask	Verified	Same for a global ask.

P-09. order types that must not take

Status: Verified

Post-only and global takers never trade; in production a crossing one fails with PostOnlyCrosses. Under the verification build require! compiles to an assumption, so the property is stated in its strongest expressible form: no reachable execution of such a taker records a trade or touches the taker's balances. Non-crossing and expired-maker paths keep the rules non-vacuous.

Rule name	Status	Description
rule_post_only_cannot_take_bid	Verified	A post-only bid taker never trades.
rule_post_only_cannot_take_ask	Verified	A post-only ask taker never trades.
rule_global_taker_cannot_take_bid	Verified	A global bid taker never trades.
rule_global_taker_cannot_take_ask	Verified	A global ask taker never trades.

P-10. “no loss of funds” for immediate-or-cancel and reverse takers

Status: Verified

The taker order types that are allowed to take preserve the market invariant through one matching step, for any maker order type other than global.

Rule name	Status	Description
rule_immediate_or_cancel_no_funds_loss_bid	Verified	IOC bid taker preserves the invariant.
rule_immediate_or_cancel_no_funds_loss_ask	Verified	IOC ask taker preserves the invariant.
rule_reverse_taker_no_funds_loss_bid	Verified	Reverse bid taker preserves the invariant.
rule_reverse_taker_no_funds_loss_ask	Verified	Reverse ask taker preserves the invariant.

P-11. “no loss of funds” for reverse makers, including coalescing

Status: Verified

A filled reverse or reverse-tight maker is placed back on the other side of the book, which debits the maker again and reserves the funds on the orderbook. When an order of the same

maker already rests at the come-back price, the new size coalesces into it. The coalesce target may sit up to one price increment away from the come-back price, the window `RestingOrder::eq` tolerates, and the rules cover that whole window. The coalesce case is where finding F-01 lives: the maker is debited the exact growth of the coalesced order's refundable allocation, computed at the target's own price, not size times the come-back price, with the added size capped so that debit never exceeds the proceeds of the fill. These rules verify the funds invariant through both the fresh-rest and the coalesce path, for `Reverse` and `ReverseTight`, on both sides, with the F-01 fix in place; that the debit stays within the proceeds is a liveness property covered by the no-revert rules in P-13.

Rule name	Status	Description
<code>rule_reverse_maker_no_funds_loss_bid</code>	Verified	Reverse maker filled by a bid taker.
<code>rule_reverse_maker_no_funds_loss_ask</code>	Verified	Reverse maker filled by an ask taker.
<code>rule_reverse_tight_maker_no_funds_loss_bid</code>	Verified	Reverse-tight maker filled by a bid taker.
<code>rule_reverse_tight_maker_no_funds_loss_ask</code>	Verified	Reverse-tight maker filled by an ask taker.
<code>rule_reverse_coalesce_bid</code>	Verified	The come-back bid of a reverse maker coalesces into an existing order within one price increment; the maker is debited the exact allocation growth (F-01).
<code>rule_reverse_coalesce_ask</code>	Verified	The come-back ask coalesces; base-side accounting is exact by construction.
<code>rule_reverse_tight_coalesce_bid</code>	Verified	Same as the bid coalesce rule for a reverse-tight maker, whose come-back price is derived with the tighter spread base.
<code>rule_reverse_tight_coalesce_ask</code>	Verified	Same as the ask coalesce rule for a reverse-tight maker.

P-12. seat pubkey integrity

Status: **Verified**

The matching code identifies the maker by the pubkey stored in the claimed-seat node, and for a global order that key decides whose global deposit pays for the fill. P-01 and P-02 therefore assume the seat node carries the key of the trader sitting in it. These rules discharge that assumption by induction: `claim_seat` establishes it, and every other verified operation preserves it. Once a seat is released its block returns to the free list and may be legitimately reused, so the invariant is stated only while the seat is held. Swap is covered by composition rather than a dedicated rule: in the verified model its only seat writes go through `claim_seat` and `update_balance`, both covered below; a dedicated swap rule trips a pointer-analysis limitation of the prover in the swap account loader (error 3003) and was left out.

Rule name	Status	Description
rule_claim_seat_writes_trader_pubkey	Verified	claim_seat writes the trader's pubkey into the seat node; claiming a second seat does not rewrite the first.
rule_seat_pubkey_preserved_by_deposit	Verified	Deposits never change stored seat pubkeys.
rule_seat_pubkey_preserved_by_withdraw	Verified	Withdrawals never change stored seat pubkeys.
rule_seat_pubkey_preserved_by_matching_bid	Verified	One matching step (bid taker, any non-global maker incl. reverse) never changes stored seat pubkeys.
rule_seat_pubkey_preserved_by_matching_ask	Verified	Same for an ask taker.
rule_seat_pubkey_preserved_by_rest_remaining_bid	Verified	Resting a bid never changes stored seat pubkeys.
rule_seat_pubkey_preserved_by_rest_remaining_ask	Verified	Resting an ask never changes stored seat pubkeys.
rule_seat_pubkey_preserved_by_cancel_bid	Verified	Cancelling a bid never changes stored seat pubkeys.
rule_seat_pubkey_preserved_by_cancel_ask	Verified	Cancelling an ask never changes stored seat pubkeys.
rule_seat_pubkey_preserved_by_release_seat	Verified	Releasing one seat does not touch the pubkey stored in the other.

P-13. no unexpected reverts on the new paths

Status: Verified

The original rule set proves that withdraw and cancel_order_by_index cannot revert for a reason other than a deliberate require! -- no overflow, no panic. The same is now shown for the paths this work adds. Like the originals, these rules assume the trade's arithmetic fits in a u64: an order too large to price is legitimately rejected, which is not an unexpected revert. The one path not covered is the swap loader, which hits the same prover pointer-analysis limitation as the swap seat rule (see What Is Not Verified).

Rule name	Status	Description
rule_place_single_order_global_no_revert_bid	Verified	Matching a global maker (taker bid) does not revert unexpectedly.
rule_place_single_order_global_no_revert_ask	Verified	Matching a global maker (taker ask) does not revert unexpectedly.

<code>rule_cancel_global_order_no_revert_bid</code>	Verified	Cancelling a global bid does not revert unexpectedly.
<code>rule_cancel_global_order_no_revert_ask</code>	Verified	Cancelling a global ask does not revert unexpectedly.
<code>rule_rest_remaining_global_no_revert_bid</code>	Verified	Resting a global bid does not revert unexpectedly.
<code>rule_rest_remaining_global_no_revert_ask</code>	Verified	Resting a global ask does not revert unexpectedly.
<code>rule_reverse_coalesce_no_revert_bid</code>	Verified	A reverse maker whose bid come-back coalesces does not revert unexpectedly; the rule bounds the come-back division and the grown-order multiply through the maker order's full value.
<code>rule_reverse_coalesce_no_revert_ask</code>	Verified	The ask-side counterpart.
<code>rule_global_deposit_no_revert</code>	Verified	Global deposit does not revert unexpectedly.
<code>rule_global_withdraw_no_revert</code>	Verified	Global withdraw does not revert unexpectedly.

What Is Not Verified

The rules above are only as strong as the model they run against. These are the places where the model is weaker than the program, in rough order of how much they matter.

Model fidelity

- **The matching step is a hand-maintained copy.** `place_single_order` in `market_helpers.rs` is the model of one iteration of the matching loop in `Market::place_order`, and the rules execute the copy, not the loop. Keeping the two identical is a manual discipline: this work found three places where the copy had already drifted (F-04). A refactor that lets the rules drive the real loop body would remove the single largest soft spot in this setup.
- **One order per side.** The mocked book holds at most one resting order per side, so matching is verified one step at a time. Walking several price levels, the ordering of the book, and the interaction between a step and the next step's state are not covered by these rules. The red-black tree that really holds the book is verified separately (`hypertree.conf`), but the two proofs are not composed.
- **Two traders, two global seats.** Balances of traders outside the model are carried as an opaque remainder inside the ghost aggregates. Properties that would need three or more distinct traders interacting are out of reach.
- **Prices are single-limb.** The verified build fixes prices to a 32 bit mantissa with four decimal places (carried over from the original assessment). Price behaviour outside that range is not covered.

Instructions and code paths not covered

- **batch_update** is verified only for the shape of what it calls: place and cancel are replaced by mocks, so the rules show it calls the right thing, not that a batch of several orders is correct end to end.
- **swap** uses a summary of `place_order` that assumes a 1:1 price, so its no-loss-of-funds rules do not cover matching at a real price. Its quoting path (`impact_base_atoms / impact_quote_atoms`) is summarized as a nondeterministic result and is not verified at all.
- **Seat pubkey preservation across swap** is argued by composition rather than proved by a rule: a dedicated rule trips a prover pointer-analysis limitation in the swap account loader (see P-12).
- **global_clean** and the **global_evict** processor are not verified. Eviction is verified at the state level (`evict_and_take_seat`, P-07); the processor around it, with its seat fee, eviction fee and token transfers, is not.
- **expand_market**, the free list and account growth are not modeled: the mock hands out fixed slots.

Arithmetic and environment

- **Token-2022 transfer fees.** SPL transfers are summarized as exact moves of the requested amount, so the "credit what actually arrived, not what was asked for" arithmetic in `process_deposit_core` and `process_global_deposit_core` is verified only in the fee-less case. Whether a mint carries a fee or hook is over-approximated as a nondeterministic choice, which does cover the security-relevant part: a transfer that will be rejected must not reduce the maker's global deposit (P-03).
- **Loops are unrolled once** (`loop_iter: 1`), which is why the matching loop is modeled a step at a time.
- **No unexpected reverts on swap is the one path left.** The no-revert property (no overflow, no panic, only deliberate require! failures) is verified for the paths this work adds, in P-13, alongside the original withdraw and `cancel_order_by_index` rules. The exception is a dedicated no-revert rule for swap, which hits the same prover pointer-analysis limitation in the swap account loader as the swap seat rule (see P-12); swap's component operations are each covered by the no-revert rules for deposit, matching and withdraw.
- **Cost and griefing are out of scope.** Nothing says an instruction fits in the compute budget, or that the book cannot be filled up or otherwise grieved.
- **Logs and return data** are compiled out of the verified build.

Appendix: Verification Run

All rule statuses in this report come from a single run of the complete rule set:

Run	Contents	Result
253e2b8ea23d4edebdb557a23c23b659	Complete rule set: 49 pre-existing rules and 52 added by this work	All 101 rules verified

Report generated July 2026. Remaining known gaps and their rationale are documented in the verification README in the repository.